

Exercise sheet 1 — Solutions

Foundations — computers, algorithms, and your first program

Published after the in-class discussion. Read these only once you have tried the sheet.

Part A — Algorithms

A1 *Write an algorithm*

No single right answer. A good one has:

- an explicit input and output;
- numbered steps;
- every quantity as a number (“30 ml”, not “a bit of”);
- every decision as a test with a yes/no answer;
- every repetition with a stated stopping condition.

Leap year, as a reference:

```
INPUT: a year y

1. IF y is not divisible by 4 THEN
    PRINT "not a leap year"; STOP
2. IF y is not divisible by 100 THEN
    PRINT "leap year"; STOP
3. IF y is divisible by 400 THEN
    PRINT "leap year"
ELSE
    PRINT "not a leap year"

OUTPUT: leap year / not a leap year
```

Worth noticing. Almost everyone writes only “divisible by 4”. 1900 was not a leap year; 2000 was. This is exactly the missing-case failure mode.

A2 *Find the ambiguity*

1. “**Look at the marks**” does nothing — it is not a step, it is a gesture.
2. “**the bad ones**” — undefined. Below 18? Below the average? The lowest one?
3. “**the number of marks**” — before or after throwing some away? These give different answers, and the algorithm does not say which.

4. “**Round**” — to how many decimals? Up, down, or to nearest?
5. **The empty case** — if every mark is “bad”, step 4 divides by zero. Nothing says what to do.

A sixth, if anyone spots it: the input is called a *list*, but no step says in what order to visit it, or that the order does not matter.

A3 Trace it

a)

step	n	i	total	
READ n	4	—	—	inputs arrive
$\text{total} \leftarrow 0$	4	—	0	
$i \leftarrow 1$	4	1	0	
$i \leq n?$ yes	4	1	0	
$\text{total} \leftarrow 0 + 1$	4	1	1	
$i \leftarrow 2$	4	2	1	
$i \leq n?$ yes	4	2	1	
$\text{total} \leftarrow 1 + 2$	4	2	3	
$i \leftarrow 3$	4	3	3	
$i \leq n?$ yes	4	3	3	
$\text{total} \leftarrow 3 + 3$	4	3	6	
$i \leftarrow 4$	4	4	6	
$i \leq n?$ yes	4	4	6	
$\text{total} \leftarrow 6 + 4$	4	4	10	
$i \leftarrow 5$	4	5	10	
$i \leq n?$ no	4	5	10	loop ends
PRINT total			10	output

b) The sum of the whole numbers from 1 to n .

c) It prints 0. The condition $1 \leq 0$ is false immediately, so the loop body never runs. Whether that is right depends on what you want — arguably 0 is the correct sum of an empty collection, but the algorithm should say so rather than leave it to luck.

Worth noticing. total starting at 0 and i starting at 1 is an **accumulator pattern**.

A4 Find the broken case

a) When $a == b$, the condition $a > b$ is false, so it goes to ELSE and prints b . The printed value happens to be correct but only by accident, because the two are equal. The real problem is that the code says “otherwise b is larger”, which is false.

The subtler answer, and the better one: the algorithm is not *wrong* for any input, but it is misleading, and the moment someone extends it (printing *which* of the two won, for example) the equal case produces a wrong statement.

b)

```
IF a > b THEN
  PRINT a
ELSE IF b > a THEN
  PRINT b
```

```
ELSE
    PRINT a, "and", b, "are equal"
```

A5 *Where it never stops*

a) Count down from 10 to 1.

b) x starts at 10 and **increases**. $x > 0$ is true at the start and gets truer. The condition never becomes false.

c) Change + to -:

```
SET x TO x - 1
```

Worth noticing. The loop condition and the update must move towards each other. This is the single most common cause of a hanging program.

A6 *Flowchart back to pseudocode*

a)

```
READ temp

IF temp > 30 THEN
    PRINT "hot"
ELSE IF temp < 10 THEN
    PRINT "cold"
ELSE
    PRINT "fine"
```

b) $\text{temp} = 30$ gives "fine" (30 is not *greater than* 30). $\text{temp} = 10$ gives "fine" (10 is not *less than* 10).

c) It is consistent, but probably not what the author meant. The boundary values belong to the middle band because both tests are strict. If you want 30 to count as hot, use $\text{temp} \geq 30$.

Worth noticing. Boundaries are where programs break. Whenever you write $<$, ask yourself deliberately whether you meant $\leq$.

A7 *Design one*

a)

```
INPUT: amount

IF amount > 100 THEN
    SET discount TO 0.20
ELSE IF amount > 50 THEN
    SET discount TO 0.10
ELSE
    SET discount TO 0

SET topay TO amount * (1 - discount)

OUTPUT: topay
```

b)

amount	discount	to pay
€40	0	€40.00
€50	0	€50.00
€75	10%	€67.50
€100	10%	€90.00
€150	20%	€120.00

c) **€100.** The rule says “more than €100”, so €100 exactly gets only the 10% band — many people write the algorithm expecting 20% there. Either the specification is sloppy or the implementation is; you have to ask which.

Worth noticing. A second trap: if you test `amount > 50` **first**, everything above €100 also matches it and you never reach the 20% branch. Order matters.

Part B — Your first Python program

B1 *Hello, world, by hand*

```
print("Hello, world")
```

a) `print` is the name of a **function** — an instruction Python already knows. The parentheses say *run it now*, and hold what we hand to it. Without them, nothing happens: you have named the function but not called it.

b) Without the quotes, Python reads `Hello, world` as two **names** rather than text, looks for variables called `Hello` and `world`, finds neither, and raises a `NameError`.

c) In the terminal, or in the output area under the notebook cell — wherever that program's standard output is connected.

B2 *Predict the output*

	expression	result
1	<code>2 ** 10</code>	1024
2	<code>17 % 5</code>	2
3	<code>17 // 5</code>	3
4	<code>2 + 3 * 4</code>	14 — multiplication first
5	<code>(2 + 3) * 4</code>	20
6	<code>10 / 2</code>	5.0 — not 5. <code>/</code> always gives a decimal
7	<code>7 // 2.0</code>	3.0 — floor division, but a decimal, because one operand was
8	<code>-7 // 2</code>	-4 — not -3. It rounds <i>down</i> , not towards zero

Number 8 catches nearly everyone. `//` floors: it goes to the nearest whole number below. For negatives that means away from zero.

B3 *Six broken lines*

	error	why	fix
1	<code>SyntaxError</code>	two arguments with no comma between them	<code>print("The answer is", 42)</code>
2	<code>NameError</code>	names are case-sensitive; <code>Print</code> does not exist	<code>print("hello")</code>
3	<code>TypeError</code>	<code>+</code> cannot join text and a number	<code>print("2 + 2 =", 2 + 2)</code>
4	<code>NameError</code>	<code>X</code> and <code>x</code> are different names	<code>print(x)</code>
5	<code>IndentationError</code>	nothing above it justifies the indent	remove the two spaces
6	<code>SyntaxError</code>	the inner <code>"</code> ends the string early	<code>print('she said "hello" and left')</code>

Number 6 is worth a minute: single and double quotes both make strings, so you can nest one kind inside the other. The alternative is the escape `\`.

Note on number 3: `print("2 + 2 = " + 2 + 2)` fails on the **first** `+`. Python evaluates left to right and never reaches the second one.

B4 *Write a small receipt*

```
print("Coffee    x 3 =", 3 * 1.50)
print("Cornetto  x 2 =", 2 * 1.50)
print("Total      ", 3 * 1.50 + 2 * 1.50)
```

Any version where the numbers are computed is correct. `print("Coffee x 3 = 4.5")` is not — that is a picture of an answer, not a program.

Worth noticing. The alignment here is done by hand with spaces, which is fragile. There is a proper way — f-strings, lecture 3.

B5 *A calculator problem*

```
change = 50 - 3 * 12.40
print("Change:", change)
print("Five-euro notes:", change // 5)
print("Left over:", change % 5)
```

Output:

```
Change: 12.799999999999997
Five-euro notes: 2.0
Left over: 2.799999999999997
```

c) Two things here look like mistakes and are not:

1. **12.799999999999997**. Not a bug in your code. Decimal fractions cannot be represented exactly in binary — the same reason $7 / 3$ ended in a stray 5. This is lecture 2, and it is why you never compare two decimals with `==`.
2. **2.0, not 2**. `//` gives a decimal when either operand is a decimal. `change` came from a calculation involving decimals, so it stays one.

B6 *Notebook or script?*

a) A notebook displays the value of the **last line of a cell** as a convenience for interactive work. A script does not display anything unless you say `print()`. The notebook behaviour is the exception, not the rule.

b) Any of these:

- a script runs top to bottom every time, so its result is reproducible;
- it is a single file you can send, schedule, import, or put under version control;
- notebook cells can be run in any order, which makes it easy to fool yourself about what state your program is actually in.